

Technical Assignment — AI Voice Bot

Backend · Supabase · Dashboard | Software Engineer Role

The goal: Build a system where a caller dials a phone number and has a natural spoken conversation with the QWR AI voice bot. The caller can choose a conversation mode (by voice or keypad), conversations are stored in Supabase with speaker-labeled transcripts, callers are associated with profiles by phone number, and a web dashboard lets our team review calls. We are primarily evaluating **voice quality, transcription accuracy, and breadth / cleanliness of integration** — not UI polish.

1. Voice Backend **CORE**

- Inbound calling via a telephony provider — Twilio, Plivo, or Exotel (your choice).
- Pipeline: speech-to-text → LLM response → text-to-speech, multi-turn with context held across the call.
- You may build this as separate STT / LLM / TTS components, *or* use a **realtime speech-to-speech API over WebRTC / streaming** — e.g. Google **Gemini Live API**, OpenAI **Realtime API**, or equivalent — as a single low-latency voice engine. Either is fine; state your choice and why. (Note: the phone caller is on a PSTN line, not WebRTC, so the realtime/WebRTC layer sits on the backend, bridging the telephony media stream to the model — don't assume the caller's phone speaks WebRTC.)
- Graceful handling of silence, "didn't catch that," and call end.

Primarily scored here:

- **Voice quality** — natural TTS or realtime model voice (justify your choice: ElevenLabs, Cartesia, Azure Neural, Google, Gemini Live, OpenAI Realtime, etc.). Streaming output (bot starts speaking before the full reply is generated) strongly preferred. Report measured latency per turn.
- **Transcription accuracy** — your STT (Deepgram, Whisper, AssemblyAI, Google) or the realtime model's transcription, how it handles Indian-accented English / Hinglish, and a **measured Word Error Rate (WER)** on a small test set you create. Measuring it at all is part of the signal.

Web search & live company data

- The bot must have **web search enabled** and use it to answer factual / current questions during the call, rather than relying only on the model's built-in knowledge.
- For questions about QWR specifically, the bot should **fetch from the company website, questionwhatsreal.com**, as its primary live source (e.g. products, industries, about). It may also reference QWR's public LinkedIn presence ([linkedin.com/company/questionwhatsreal](https://www.linkedin.com/company/questionwhatsreal)) where useful.
- *Note: LinkedIn blocks most automated/server-side fetching and scraping it violates their terms. Treat the website as the reliable source; if you use LinkedIn data, use the public profile information and note in your README how you handled the fetch limitation. Honestly documenting what is and isn't fetchable counts in your favor.*

Call / network quality detection **OPTIONAL**

Many calls in India land on 2G / GSM, where the carrier's narrowband codec degrades the audio before it ever reaches the telephony provider. Rather than trying to fix that, implement **call / network quality detection and adaptation**. You can't read the caller's signal bars, but your backend does see usable signals — STT confidence trending low, a rising retry / mis-recognition rate, packet loss or jitter on the media stream, the negotiated codec (AMR-NB / GSM vs. wideband), and round-trip latency. From those, the bot adapts its behavior:

- Slow down and shorten prompts when quality drops.
- Proactively offer **keypad (DTMF) input** — this is where the dual-input requirement pays off: a recognition failure becomes a graceful recovery, not a dead call.
- Repeat back what it heard for confirmation when confidence is low.
- Acknowledge a poor line naturally ("the line sounds rough — want to use the keypad?").

Optional — a nice-to-have that signals depth. If you implement it, note your approach in the README. If you don't, a short paragraph on how you would approach it is still worth including.

2. Mode Selection **CORE**

The bot offers four conversation modes the caller can choose from. Early in the call, the bot should let the caller pick one **by voice** (e.g. "Challenge mode") *and* **by keypad** (e.g. press 2). The selected mode changes how the bot behaves for the rest of the call.

- **1 — Think:** a thinking partner; follows the thread, has opinions, can riff.
- **2 — Challenge:** pressure-tests the caller's idea; pushes back, asks the hard questions.
- **3 — Explore:** voice-based learning; explains a topic conversationally in short chunks.
- **4 — Guide:** a steadier, slower pace for working through a decision or something difficult.

*Each mode must be backed by an **injected mode prompt** — a behavioral instruction block, specific to the selected mode, added to the system prompt so the bot's posture actually changes (not just a label). Condensed reference definitions for the four modes are in the Appendix; you may use them as-is or adapt them. What matters is that selecting a mode injects its prompt and the behavior shifts accordingly. The selected mode should also be stored against the call record in Supabase.*

Dual input — required on every option question: Any question in the call that offers the caller a choice (mode selection, the known-caller / anonymous prompt, yes/no confirmations, etc.) **must accept both spoken answers and DTMF keypad input**. The caller can always either say the option or press the corresponding key. Handle no-input and unrecognized-input gracefully (re-prompt, then fall back) rather than dropping the call.

3. Conversational Intake & Extraction

- For unknown callers, the bot runs a short, natural intake — name plus a few useful context details (company / role, city, reason for calling).
- Details must be **extracted from natural speech into structured fields** — not stored as raw text blobs. The caller speaks freely; your system parses values into columns.
- Extracted data is **validated / normalized** before storage: E.164 phone format, no hallucinated / invented values, unanswered fields left *null* rather than guessed.

- The stored profile then **personalizes the rest of the conversation** (the bot uses the name / context in later turns).
- State in your README which extraction approach you chose — scripted slot-filling, LLM structured output, or a hybrid — and why.

4. Supabase Data Layer

- Store each call: metadata (number, timestamp, duration, selected mode, profile link) and the full transcript with **per-turn speaker labels** (caller vs bot).
- Schema is yours to design — expect at least *profiles*, *calls*, and *transcript_turns* (or messages).
- **Caller identity flow:** on call start, match the caller's E.164 number against the profiles table.
 - **Known caller** → greet by name, link the call to that profile.
 - **Unknown caller** → offer to collect details (create / enrich a profile) *or* proceed in **anonymous mode**; the call is still stored, linked to a number-only or anonymous record.
- **Diarization:** turn-level speaker labeling (caller vs bot) is required. If you implement true multi-speaker diarization, note the tool used (e.g. Deepgram *diarize*, pyannote) and why it's warranted.

5. Dashboard

- Web UI showing live / active calls and a history of past calls.
- Per-call detail: speaker-labeled transcript, duration, linked profile (or “anonymous”), and ideally the recording plus per-turn latency.
- Transcript search / filter (Supabase Postgres full-text search is a good fit).

6. Session Summary Report

- When the caller asks for a summary during the call, *or* when the session ends, the bot generates a **summary of the conversation** (key points, any details captured, outcome) from the stored transcript in Supabase.
- The summary is **delivered to the caller** via one of: **email**, **SMS**, or **WhatsApp** — either as the summary text itself or as a **link** to that call's detail page in the dashboard.
- At minimum, implement **email or SMS** reliably. Email via any transactional provider (Resend, SendGrid, SES); SMS via your telephony provider (Twilio / Plivo).
- *Note on WhatsApp: business-initiated WhatsApp messages outside the 24-hour service window require an approved Cloud API number and a pre-approved message template — not realistic to fully set up for this assignment. WhatsApp is therefore optional / best-effort; if you attempt it, note in your README how far you got and what the gating requirements are. Email or SMS satisfies the requirement.*
- The contact destination (email / phone) should come from the caller's profile or be captured during the call, and stored with the call record.

Architecture We're Looking For

A clean separation: a **core voice engine** (STT → LLM → TTS) decoupled from the telephony adapter and from the dashboard. The dashboard should read from Supabase and never sit in the call path. If logging blocks the call, or everything is one tangled service, that counts against the submission.

Deliverables

- A dial-in number we can call (or a recorded call if you prefer not to keep a paid number running), and a sample summary delivered by email or SMS.
- The dashboard — deployed link or screen recording.
- The Supabase schema (SQL export or screenshot).
- A GitHub repository with a README covering: architecture + ER diagram, STT / TTS choices and rationale, measured latency and WER, your extraction and diarization decisions, and any tradeoffs.

Evaluation Rubric

Criterion	Weight
Voice naturalness & latency	20%
Transcription accuracy & how it was measured (WER)	15%
Mode selection + injected mode prompts + voice/keypad (DTMF) dual input	20%
Web search enabled + live company data from questionwhatsreal.com	5%
Supabase schema & speaker-labeled storage	15%
Caller identity / anonymous flow & extraction accuracy	10%
Session summary report (email / SMS, or link)	10%
Dashboard (live + history + search)	5%
Decoupled architecture & README clarity	5%

Logistics

- **Estimated completion time:** Reply with your own honest estimate of how long you'll need *before* you begin.
- **Paid plans / phone access:** If any part of the phone-calling functionality requires a paid plan or credits (e.g. a Twilio / Plivo number, per-minute charges), tell us what you need — we'll arrange the access or credits rather than have you pay out of pocket.
- **Identity / address proof:** Please include a copy of a government-issued identity or address proof with your submission, for our records.
- **Next steps:** Once the assignment is completed and cleared, we'll send the offer letter and request any further documentation.

QWR fact sheet (for the bot to answer from — extend as you like):

QWR Interactive Solutions is a Pune-based deep-tech company that designs and manufactures head-worn computers — VR headsets and AI smart glasses — shipped globally to enterprises and consumers. “QWR” stands for “Question What’s Real.” The bot should be able to answer basic questions about who QWR is, what it builds, and where it’s based, then steer the caller back to the intake / conversation.

Appendix — Reference Mode Prompts (injectable)

Condensed behavioral blocks for each mode. Inject the block for the selected mode into the system prompt. Use as-is or adapt; the exact wording is not graded, the behavioral shift is.

1 — THINK

A thinking partner — a sharp, curious friend you can think out loud with. Engage with the substance of what the caller says: react, agree, disagree, build on it, share real opinions with reasoning. Ask a question only when you genuinely need more to be useful, one at a time. Don't bounce every statement back as a question; don't play therapist. Be brief when they're brief, expand when they expand.

2 — CHALLENGE

Devil's advocate, on the caller's side but refusing to nod along. Pressure-test their claim with real, specific objections drawn from what they actually said. Ask for evidence; surface hidden assumptions; steelman the counter-argument. Concede honestly when they make a good point ("Fair."), then ask the next harder question. Don't nitpick for cheap points, don't be contrarian forever, don't get hostile.

3 — EXPLORE

Voice-based learning. The caller names a topic; explain it conversationally in 90-second to 2-minute chunks, then pause and check in ("Want me to go deeper, or shift?"). Use analogies and concrete examples, not jargon or bullet-point delivery. Admit what's contested vs settled, and say "honestly, I don't know" rather than bluffing. The one mode where you talk more than you listen — but never for more than two minutes without checking in.

4 — GUIDE

A steadier, slower hand — for working through a decision between options, or sitting with something difficult. Slow the pace, offer structure when the caller is overwhelmed, gently name the shape of the problem ("It sounds like X matters more to you than Y"). You may offer a frame, but do not give direct advice ("you should do X" is out). Acknowledge difficulty briefly, then keep moving. Walk alongside; don't lead, don't decide for them.